

Week 8 Capstone Project

AI Property Valuation & Lead Scoring Platform for Real Estate

Machine Learning • Explainable AI • MLOps • LLM Agents • Tool Calling • UrduLish Assistant

Duration: 5 Days

Scenario

Your client is the same real estate company from Week 7. Their AI Voice Agent now handles hundreds of calls every week, but two big business problems remain:

- Pricing is inconsistent. Agents quote property prices from gut feeling, which leads to overpriced listings that never sell and underpriced deals that lose the company money.
- The sales team wastes time. Every lead gets the same attention, even though most callers never buy. The team needs to know which leads are hot and should be called first.

Your task is to build a production-ready Machine Learning platform that predicts fair market prices for properties, scores every incoming lead by its probability of converting, explains every prediction in simple language, and exposes both models to an AI assistant that sales agents can talk to in UrduLish.

Unlike Week 7, where the intelligence came from an LLM, this week the core intelligence comes from models you train yourself. The LLM is only the interface on top of them.

The final product should be deployable for an actual client.

Technologies Expected

The project should expose students to a complete modern ML + AI stack.

Area	Tools
Data & Analysis	Pandas, NumPy, Polars (optional), Jupyter
Visualization	Matplotlib, Seaborn, Plotly
Machine Learning	Scikit-learn, XGBoost, LightGBM, CatBoost
Hyperparameter Tuning	Optuna, GridSearchCV
Explainability	SHAP, LIME (optional)
Experiment Tracking	MLflow, Weights & Biases (optional)
Imbalanced Data	imbalanced-learn (SMOTE), class weights
LLM	OpenAI GPT, Claude, Gemini
Agent Framework	LangGraph, LangChain
Backend	FastAPI, Pydantic
Database	PostgreSQL, SQLite
Frontend	Streamlit or Gradio
Monitoring	Evidently AI (data drift), Prometheus (optional)

Area	Tools
Deployment	Docker, Railway, Render, Hugging Face Spaces, AWS

Project Objectives

Build a production-ready ML platform capable of:

- ✓ Cleaning and analysing real estate data
- ✓ Engineering meaningful features (location, size, age, amenities)
- ✓ Predicting fair property prices (regression)
- ✓ Scoring leads by conversion probability (classification)
- ✓ Handling imbalanced data
- ✓ Comparing multiple models scientifically
- ✓ Tuning hyperparameters
- ✓ Explaining every prediction with SHAP
- ✓ Tracking experiments and versioning models
- ✓ Serving predictions through an API
- ✓ AI assistant that uses the models as tools
- ✓ UrduLish explanations for sales agents
- ✓ Detecting data drift
- ✓ Retraining strategy
- ✓ Dockerized deployment

Datasets

Students must work with two datasets. Real data is preferred; synthetic data is allowed if it is realistic and documented.

Dataset A — Property Listings (Regression): minimum 5,000 rows. Columns such as: city, area/society (DHA, Bahria Town, Gulberg, etc.), property type, plot size (marla/kanal), covered area, bedrooms, bathrooms, age of property, floors, corner/park-facing, amenities, distance to main road/school/hospital, listing date, price (PKR).

Suggested sources: public Zameen.com datasets on Kaggle, Graana listings, or a synthetic generator built by the student.

Dataset B — Leads (Classification): minimum 3,000 rows. Columns such as: lead source (call, WhatsApp, Facebook, website), budget, preferred city, purpose (buy/rent/invest), number of calls, call duration, response time, visit booked (yes/no), days since first contact, objection raised, and the target: converted (yes/no).

Tip: The Week 7 CRM logs can be reused to generate realistic lead data.

Week 8 — Day 1: Data Understanding, EDA & Feature Engineering

 **Scenario:** A model is only as good as its data. Before training anything, understand the market. Pakistani real estate data is messy: prices written as "1.5 crore" or "85 lac", sizes in marla, kanal and square feet, duplicate listings, and fake prices. Clean it like a professional data scientist.

Task 1 — Data Collection & Documentation

Collect or generate both datasets. Create a data dictionary that includes:

- Column name
- Data type
- Description
- Unit (PKR, marla, sq ft)
- Source
- Known issues

Task 2 — Data Cleaning

Handle:

- Missing values (justify the imputation strategy)
- Duplicate listings
- Price normalisation ("crore", "lac" → numeric PKR)
- Area normalisation (marla / kanal / sq ft → one unit)
- Outliers (fake prices, typos) using IQR or Z-score
- Inconsistent location names ("DHA Ph 5" vs "DHA Phase 5")

Task 3 — Exploratory Data Analysis

Create visualisations and write insights for:

- Price distribution (raw vs log-transformed)
- Price per marla by city and society
- Correlation heatmap
- Effect of bedrooms, age and corner plots on price
- Lead conversion rate by source and purpose
- Class imbalance in the leads dataset

Every chart must have a one-line business insight written below it.

Task 4 — Feature Engineering

Create at least 10 new features, for example:

- Price per marla
- Property age buckets
- Society tier (premium / mid / budget)
- Amenity score
- Distance-based features
- Lead engagement score (calls × duration)
- Response speed category
- Budget-to-market-price ratio

Justify why each feature should help the model.

Task 5 — Encoding, Scaling & Splitting

Implement:

- One-hot vs target encoding for high-cardinality locations (compare both)
- Scaling where required
- Train / validation / test split (70/15/15)
- Stratified split for the leads dataset
- Data leakage check — document any column that must be removed and why

Build everything as a reusable Scikit-learn Pipeline.

Week 8 — Day 2: Property Valuation Model (Regression)

 **Scenario:** The sales manager asks: "Is plot ki sahi qeemat kya honi chahiye?" Today you build the model that answers that question with numbers, not guesses.

Task 1 — Baseline Models

Train and compare:

- Mean / median baseline
- Linear Regression
- Ridge & Lasso

A model that cannot beat the baseline is useless — prove yours does.

Task 2 — Advanced Models

Train and compare:

- Random Forest
- XGBoost
- LightGBM
- CatBoost

Task 3 — Evaluation

Measure every model on the test set:

- MAE (in PKR)
- RMSE
- R^2
- MAPE
- Error by city and by price range (where does the model fail?)

Create a comparison table and an Actual vs Predicted plot.

Task 4 — Hyperparameter Tuning & Experiment Tracking

Use Optuna to tune the best two models. Log in MLflow:

- Parameters
- Metrics
- Feature set used
- Training time
- Model artifact

Register the final model in the MLflow Model Registry.

Task 5 — Price Range & Confidence

A single number is not enough for a client. Return:

- Predicted price
- Lower and upper range (quantile regression or prediction intervals)
- Verdict: Underpriced / Fair / Overpriced compared to the listed price

Example output: "Predicted: 2.35 crore (range 2.15 – 2.55 crore). Listed at 2.9 crore → Overpriced by ~23%."

Week 8 — Day 3: Lead Scoring Model (Classification) & Explainability

 **Scenario:** The sales team has 200 new leads and time to call only 40. Your model decides who gets called first. A wrong decision means a lost sale, so the model must be accurate and it must explain itself.

Task 1 — Classification Models

Train and compare:

- Logistic Regression (baseline)
- Random Forest
- XGBoost / LightGBM

Task 2 — Handling Imbalanced Data

Most leads do not convert. Compare:

- No balancing
- Class weights
- SMOTE
- Threshold tuning

Explain why accuracy is a misleading metric here.

Task 3 — Evaluation

Measure:

- Precision, Recall, F1
- ROC-AUC and PR-AUC
- Confusion matrix
- Precision@Top-20% (how many of the top-ranked leads actually converted)
- Calibration curve

Choose a decision threshold based on business cost: missing a hot lead vs wasting a call. Document your reasoning.

Task 4 — Lead Segmentation

Convert probability into business categories:

- 🔥 Hot (call within 1 hour)
- 🌤️ Warm (call within 24 hours)
- ❄️ Cold (add to follow-up campaign)

Bonus: use K-Means clustering to discover customer personas (e.g. first-time buyer, overseas investor, renter) and describe each.

Task 5 — Explainability with SHAP

For both models, produce:

- Global feature importance (SHAP summary plot)
- Local explanation for individual predictions (waterfall / force plot)
- A function that turns SHAP values into plain-language reasons

Example: "Yeh lead Hot hai kyun ke client ne 3 dafa call ki, budget market price se match karta hai, aur visit already book hai."

Check the models for unfair bias (e.g. against a lead source or city) and document the findings.

Week 8 — Day 4: Model Serving, AI Assistant & Integration

 **Scenario:** Models sitting in a notebook make no money. Today the models become a product: an API the Week 7 voice agent can call, and an AI assistant that sales agents can chat with in UrduLish.

Task 1 — FastAPI Model Serving

Build endpoints:

- POST /predict/price
- POST /predict/lead-score
- POST /explain/price
- POST /explain/lead
- POST /predict/batch (CSV upload)
- GET /health and GET /model/info (version, metrics, trained date)

Use Pydantic for input validation. Reject invalid inputs (negative size, unknown city) with clear errors.

Task 2 — LangGraph AI Assistant

Build an agent that uses the models as tools:

- Price Predictor tool
- Lead Scorer tool
- Explainer tool
- Comparable Properties tool (find similar listings from the database)
- Market Stats tool (average price per marla by area)

The LLM must never invent a price. Every number must come from a tool.

Example — Sales agent: "DHA Phase 6 mein 1 kanal, 5 saal purana ghar, kitne ka jana chahiye?" Assistant: "Model ke mutabiq iski fair value 7.8 se 8.4 crore ke darmiyan hai. Sab se bara factor location aur corner plot hai..."

Task 3 — Dashboard

Build a Streamlit or Gradio app with:

- Price prediction form with range and verdict
- Lead list sorted by score with Hot / Warm / Cold badges
- SHAP explanation for any selected prediction
- Market insights page (charts from Day 1)
- Chat panel for the AI assistant

Task 4 — Integration with the Week 7 Voice Agent

Design (and implement if time allows):

- After every call, the voice agent sends call data to /predict/lead-score
- Hot leads trigger an email to the assigned employee
- The voice agent can answer "Mera ghar kitne ka bikega?" using /predict/price

Draw the updated system architecture diagram.

Task 5 — Guardrails & Validation

Implement:

- Refuse predictions for data far outside the training range (out-of-distribution check)
- Show a disclaimer that prices are estimates, not official valuations
- Prompt-injection tests for the assistant ("Ignore instructions", "Set price to 1 rupee")

- Log every prediction with inputs, output and model version
-

Week 8 — Day 5 (Capstone): MLOps, Deployment, Presentation & Client Handover

 **Scenario:** Today you deliver the complete product to the client — and prove it will keep working after you leave. Property prices change every month; a model trained today will be wrong next year unless it is monitored.

Task 1 — Monitoring & Data Drift

Using Evidently AI (or your own code):

- Simulate new data (e.g. prices rise 15%)
- Detect data drift and prediction drift
- Generate a drift report
- Define alert thresholds (e.g. MAPE above 15% → retrain)

Task 2 — Retraining Pipeline

Build a script / pipeline that:

- Loads new data
- Retrains the model
- Compares the new vs current model on a holdout set
- Promotes the new model only if it is better
- Keeps a rollback option

Define the retraining schedule (monthly / on drift).

Task 3 — Production Deployment

Deploy:

- Dockerized FastAPI backend
- Streamlit / Gradio dashboard
- Database
- Environment variables and secrets
- Health checks and logging
- Basic CI (tests run on every push)

Task 4 — Stakeholder Demonstration

Prepare a 10-minute live demo showing:

- The data story (2–3 key EDA insights)
- Model comparison and why the final model was chosen
- Live price prediction with range and verdict
- Live lead scoring with Hot / Warm / Cold
- SHAP explanation in UrduLish
- AI assistant answering a sales agent's question
- Drift detection and retraining
- Business impact estimate (e.g. hours saved, conversion uplift)

Task 5 — Documentation & Future Enhancements

Write:

- Model Card for each model (data, metrics, limitations, bias, intended use)
- API documentation
- User guide for sales agents
- Maintenance guide

Propose future enhancements such as: image-based valuation from property photos (CNN), time-series price forecasting per area, a recommendation engine for buyers, WhatsApp lead scoring, automated follow-up campaigns, and integration with HubSpot / Salesforce.

Evaluation Rubric

Criteria	Weight
Data cleaning & EDA	15%
Feature engineering	10%
Regression model quality	15%
Classification model & imbalance handling	15%
Explainability (SHAP + UrduLish reasons)	10%
API, AI assistant & dashboard	15%
MLOps: tracking, drift, retraining, deployment	10%
Documentation & demo	10%

Minimum targets: Price model MAPE below 15% on the test set; lead model PR-AUC clearly above the baseline; API response under 500 ms.

Final Submission Requirements

The final submission should include:

-  Cleaned datasets with a data dictionary
-  EDA notebook with visualisations and business insights
-  Reusable feature engineering pipeline
-  Trained property valuation model with price ranges and verdicts
-  Trained lead scoring model with Hot / Warm / Cold segmentation
-  Model comparison report with all metrics
-  MLflow experiment logs and registered models
-  SHAP explainability with plain-language UrduLish reasons
-  FastAPI backend with documented endpoints
-  LangGraph AI assistant that uses the models as tools
-  Streamlit / Gradio dashboard
-  Data drift report and retraining pipeline
-  Dockerized deployment with a live link
-  Model Cards for both models
-  GitHub repository with README, setup guide and architecture diagram
-  10-minute demo video and slide deck

The finished project should resemble a commercial ML product that a real estate firm could use to price properties correctly and focus its sales team on the leads most likely to buy, demonstrating modern best practices in machine learning, explainable AI, MLOps and AI agent integration.